

依頼要約

「公文書自動フォルダ分けシステム」のコア解析ロジック（OCR取得 → OllamaによるSLM解析 → Pydantic型検証/確信度計算/隔離判定）を `app/services/` 配下の実装してください。

前提条件・制約事項

- **Python:** 3.10+
- **ライブラリ:** azure-ai-formrecognizer (Azure AI Document Intelligence用), httpx (Ollama REST API呼び出し用), pydantic v2
- **設定参照:** app.core.config.settings より環境変数を読み込むこと
- **セキュリティ & 堅牢性:**
 - APIキーやエンドポイントのハードコード厳禁
 - SLMの応答が不正なJSONであってもクラッシュせず、エラーをハンドリングして `needs_review=True` として処理を継続すること

作成・更新するファイル

```
backend/app/
├── services/
│   ├── __init__.py
│   ├── ocr_service.py      # Azure AI Document Intelligence 連携
│   ├── slm_service.py     # Ollama (Llama-3-8B等) 連携
│   ├── validation_service.py # 確信度計算・Pydantic検証・隔離判定
│   └── pipeline_service.py # パイプライン統括サービス
```

各ファイルの要件仕様

1. app/services/ocr_service.py

Azure AI Document Intelligence (DocumentAnalysisClient) を使用してPDF/画像からテキストを抽出します。

- **クラス名:** OCRService
- **メソッド:** `async def analyze_document(self, file_bytes: bytes) -> str`
 - 受け取ったバイト列を `begin_analyze_document("prebuilt-read", ...)` に渡して非同期/同期で処理（`asyncio.to_thread` 等を活用してFastAPIのイベントループをブロックしないこと）。
 - 各ページ・各行のテキストを1つの整形された文字列として結合して返却。
 - エラー発生時は適切にログ出力し、例外をレイズ。

2. app/services/slm_service.py

Ollamaの `http://api/generate` または `/api/chat` に対して `httpx.AsyncClient` を使ってリクエストを送ります。

- クラス名: `SLMService`
- メソッド: `async def extract_metadata(self, ocr_text: str) -> dict`
 - システムプロンプト: 行政文書の「文書種別(`document_type`)」「発行年月日(`issue_date: YYYY-MM-DD`)」「文書番号(`document_number`)」「差出人(`sender`)」「宛先(`recipient`)」を抽出するよう厳格に指示。必ず純粋なJSONオブジェクトのみを出力させる (`format="json"` オプションを使用)。
 - レスポンス処理: 返却されたJSON文字列を `json.loads` で辞書化して返却。パース失敗時はエラーログを出し空辞書 `{}` または部分結果を返却。

3. app/services/validation_service.py

SLMから返ってきた辞書データを評価し、Pydanticモデル `DocumentMetadata` へ変換・検証します。

- クラス名: `ValidationService`
- メソッド: `def validate_and_score(self, raw_data: dict, ocr_text: str) -> DocumentMetadata`
 - 確信度 (**Confidence Score**) 計算ロジック:
 - 基本スコア 1.0 からスタート。
 - 必須項目 (`document_type`, `issue_date`) が欠損または `None` の場合: スコア -0.3
 - `issue_date` が `YYYY-MM-DD` 形式に正規化できない場合: スコア -0.2
 - `document_number`, `sender`, `recipient` の欠損ごとに: スコア -0.1
 - SLMが抽出したテキスト (例: 差出人名や日付) が `ocr_text` 内に存在しない (幻覚/ハルシネーションの可能性) 場合: スコア -0.2
 - スコアを 0.0 ~ 1.0 の範囲にクリップ。
 - 要確認フラグ (**needs_review**) 判定:
 - 計算された `confidence_score < settings.CONFIDENCE_THRESHOLD` (デフォルト 0.85) の場合、または `document_type` や `issue_date` が取得できない場合は `needs_review = True` に設定。

4. app/services/pipeline_service.py

上記の3つのサービスを統合し、ファイルアップロードから最終的な `DocumentMetadata` 生成までの流れを一貫して処理します。

- クラス名: `DocumentPipelineService`
- メソッド: `async def process_document(self, file_bytes: bytes, filename: str) -> DocumentProcessResponse`
 - 1. OCR実行 (`ocr_service`)
 - 2. SLM解析実行 (`slm_service`)

-
- 3. 型検証・確信度計算 (validation_service)
-
- 4. 仮の移動先パス（例: needs_review=True なら /storage/NEEDS_REVIEW/filename、False なら /storage/APPROVED/document_type/filename）を生成。
-
- 5. 結合された DocumentProcessResponse を返却。

出力フォーマット

上記4ファイルの完全なPythonコードを、ファイルパスを明記したコードブロック形式で出力してください。プレースホルダーや省略（# TODO 等）を一切使わず、そのまま動作する完全なコードを生成してください。